UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 9 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Computer generations and history

Source: CSC429 (10).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Y technology is the transistor. The second generation of computers (roughly 1956–1963) replaced vacuum tubes with transistors, which were smaller, faster, more reliable and generated far less heat; early transistorised machines such as those used in the atomic energy industry are characteristic of this generation.

(b) Machine language is the set of binary instructions (0s and 1s) that the CPU executes directly; it is machine-dependent, uses opcodes and operands in binary, requires no translation, and is difficult for humans to read or write. Assembly language is a low-level symbolic representation of machine language in which mnemonics (e.g. ADD, MOV) and symbolic names replace binary opcodes and addresses; it is still machine-specific but far more readable, and it must be translated into machine code by an assembler before execution. In short, machine language is the binary form executed directly by hardware, whereas assembly language is its human-readable mnemonic counterpart requiring assembly.

Unofficial marking guide:

- *1 mark: (a) correctly identifies Y as the transistor.*
- *1 mark: (b) states machine language is binary/0s and 1s executed directly by the CPU.*
- *1 mark: (b) states assembly language uses mnemonics/symbolic names and is a low-level symbolic form of machine language.*
- *1 mark: (b) notes machine language needs no translation while assembly language requires an assembler.*
- *1 mark: (b) notes both are machine-dependent/low-level, with a clear contrast in readability and human usability.*

Model confidence: high.

Model note: Assumes Y = transistor, consistent with the standard second-generation description; no figure required.

QUESTION A2 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The technique is direct addressing (also called absolute addressing). The instruction's address field A holds the actual/effective address of the operand in memory, so the operand is fetched directly from memory location A — no intermediate calculation of the effective address is needed.

(b) How it is used: The instruction is fetched and its opcode decoded; the CPU takes the address field A as the effective address (EA = A) and places it on the address bus; the memory is read and the operand stored at location A is transferred to the CPU (e.g. into a register or the ALU). For example, in LOAD A, the processor reads the contents of memory address A and loads them into the accumulator. The operand is thus accessed in a single memory reference after the instruction fetch.

(c) Advantage: It is simple and fast — only one memory access is needed to obtain the operand (no extra address calculation or indirection), and no special addressing hardware is required. Disadvantage: The address field must be large enough to hold a full memory address, limiting the range of addressable memory (and the number of bits left for the opcode); also the instruction is not relocatable and offers limited flexibility for arrays/pointers.

Unofficial marking guide:

- *1 mark: Correctly names the technique as direct addressing (absolute addressing).*
- *1 mark: States that the address field A is the effective address of the operand (EA = A).*
- *1 mark: Explains the fetch/execute step — address A is sent to memory and the operand at A is read into the CPU/register.*
- *1 mark: Gives one valid advantage (e.g. simple, fast, single memory access, no extra address calculation).*
- *1 mark: Gives one valid disadvantage (e.g. limited addressable range / large address field needed, inflexible, not relocatable).*

Model confidence: high.

Model note: Figure 2 is described only by its extracted text (Instruction: Opcode | Address A; Memory: Operand). The answer assumes the standard interpretation that Address A points directly to the operand's memory location. If the figure actually shows an

© Hak Cipta Universiti Teknologi MARA

arrow from Address A to a memory location containing the operand's address (i.e. indirect addressing), the label in (a) would change to indirect addressing; this should be confirmed from the figure.

QUESTION A3 (5 marks)

Topic: Data representation and number systems

Answer: 2.510 = 0x40200000 (sign=0, exponent=10000000, fraction=01000000000000000000000)

- *1 mark for the sign bit.*
- *1 mark for the biased exponent.*
- *1 mark for the 23-bit fraction.*
- *1 mark for the correct 32-bit grouping.*
- *1 mark for the final hexadecimal value.*

QUESTION A4 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (8).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Classification of parallel processors is constructed according to the relationship between the instruction stream and the data stream, as proposed in Flynn's taxonomy. A parallel processor is described by whether it issues one or many instruction streams and whether it operates on one or many data streams, giving four categories: SISD (single instruction, single data), SIMD (single instruction, multiple data), MISD (multiple instruction, single data), and MIMD (multiple instruction, multiple data). This classification is constructed by observing how instructions and data are organised and processed concurrently in the system.

(b) One example for each parallel processor architecture: SISD – a conventional single-core uniprocessor (e.g., a basic scalar CPU); SIMD – a vector processor or GPU (e.g., Intel SSE/AVX vector unit, or a GPU shader array); MISD – a systolic array (e.g., a systolic array for matrix multiplication); MIMD – a multicore processor or a cluster of workstations (e.g., Intel Core i7 multicore, or a Beowulf cluster).

Unofficial marking guide:

- *1 mark: Stating that classification is based on instruction and data streams (Flynn's taxonomy).*
- *1 mark: Naming the four categories (SISD, SIMD, MISD, MIMD) with a brief description of each.*
- *1 mark: Correct example for SISD.*
- *1 mark: Correct example for SIMD.*
- *1 mark: Correct example for MISD and MIMD (one mark for both, or 0.5 each).*

Model confidence: high.

Model note: Assumes the question refers to Flynn's taxonomy, which is the standard classification for parallel processors in computer organization. Examples are illustrative; other valid examples are acceptable.

QUESTION A5 (10 marks)

Topic: Data representation and number systems

a) Answer: left 28 -> 0 0011100; right -78 -> 1 1001110; 28 - (-78) = 106 -> sign-magnitude 0 1101010

- *1 mark for operand representations.*
- *1 mark for sign handling.*
- *1 mark for magnitude arithmetic.*
- *1 mark for the result sign bit.*
- *1 mark for the final byte.*

b) Answer: left 28 -> 1's 00011100, 2's 00011100; right -78 -> 1's 10110001, 2's 10110010; 28 - (-78) = 106 -> 1's 01101010, 2's 01101010

- *1 mark for the positive operand in binary.*
- *1 mark for the 1's complement of the negative operand.*
- *1 mark for adding 1 to form the 2's complement.*
- *1 mark for the correct 8-bit result.*
- *1 mark for converting the result back to decimal.*

© Hak Cipta Universiti Teknologi MARA

QUESTION A6 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (5).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Direct mapping does not need a replacement algorithm. In direct mapping each main-memory block maps to exactly one fixed cache line (line = block number mod number of lines), so when a new block must be loaded there is only one possible line it can occupy. That line is simply overwritten; no choice between candidate lines exists, hence no replacement policy is required. (By contrast, fully associative and set-associative caches have several candidate lines/blocks in a set and therefore need a replacement algorithm.)

(b) Four replacement algorithms:
1. Random replacement — a line is chosen at random from the candidate set/block to be evicted. It is simple and cheap to implement in hardware, requires no bookkeeping, but gives no regard to locality and can evict a heavily used block, so hit ratio is generally poorer.
2. FIFO (First-In, First-Out) — the block that has been resident in the cache the longest is replaced. It is easy to implement with a queue/circular buffer and a pointer, but it ignores how recently or how often a block is used, so a frequently used old block may be evicted (no account of temporal locality).
3. LRU (Least Recently Used) — the block that has not been referenced for the longest time is replaced, exploiting temporal locality. It gives good hit ratios but requires tracking of usage order (e.g. counters or a stack), which is costly in hardware, especially for large associativity.
4. LFU (Least Frequently Used) — the block with the smallest reference count is replaced, using a counter per block. It favours frequently used blocks but old counts are not aged, so blocks that were once hot can linger; also needs counters and comparison logic.
(Other acceptable: NRU/clock, optimal/OPT as a theoretical benchmark.)

Unofficial marking guide:

• *2 marks: (a) correctly identifies direct mapping and explains that each block has only one fixed cache line so it is simply overwritten (1 mark for naming direct mapping, 1 mark for justification).*

• *8 marks: (b) 2 marks per replacement algorithm × 4 = 8 marks; each algorithm earns 1 mark for correct naming and 1 mark for an accurate description of how it selects the victim (and/or its advantage/disadvantage).*

Model confidence: high.

Model note: Question text is clear; no OCR correction needed. Part (b) accepts any four standard algorithms (Random, FIFO, LRU, LFU, and also NRU/clock or OPT).

QUESTION A7 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (3).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Three types of system interconnection [6 marks]

1. Bus interconnection: A set of shared parallel conducting lines (address bus, data bus and control bus) that connects the CPU, main memory and I/O peripherals. Because all devices share the same lines, only one transfer can take place at a time; the bus is simple and cheap but becomes a bottleneck as more devices are added. Performance can be improved by using separate buses or multiple bus hierarchies (e.g. a fast local/cache bus plus a slower system bus).

2. Point-to-point interconnection: A dedicated link connects each pair of components (or each device to a central switch), so multiple transfers can occur simultaneously and each link can be tuned to the speed of the devices it joins. It gives higher bandwidth and better scalability than a shared bus, at the cost of more wiring/complexity. Examples include switched interconnect fabrics and serial point-to-point links such as PCI Express.

3. Interconnection network (multistage/switch-based network): Components are joined through a network of switches and links, e.g. a crossbar switch or a multistage interconnection network (omega/butterfly network). This allows many simultaneous paths between processors, memory modules and I/O, giving high throughput and fault tolerance for multiprocessor and large systems, but requires more hardware and control logic.

(Other acceptable answers: shared-bus vs. switched fabric; synchronous vs. asynchronous timing of the interconnection; centralized vs. distributed interconnection.)

© Hak Cipta Universiti Teknologi MARA

(b) Two sub-components of the CPU and their functions [4 marks]

1. Control Unit (CU): Directs and coordinates the operation of the CPU and the rest of the computer. It fetches and decodes instructions, generates the timing and control signals that tell the ALU, registers and memory/IO what to do, and sequences the steps of the instruction cycle (fetch-decode-execute).

2. Arithmetic Logic Unit (ALU): Performs the actual data processing — arithmetic operations (add, subtract, multiply, divide) and logical operations (AND, OR, NOT, XOR, shift, compare) — on operands supplied from registers, and sets condition/status flags (zero, carry, sign, overflow) in the status register.

(Also acceptable: registers such as the Program Counter, Instruction Register, Memory Address Register, Memory Buffer Register and general-purpose registers, with their storage/pointing functions.)

Unofficial marking guide:

- *1 mark: naming the first type of system interconnection (e.g. bus interconnection)*
- *1 mark: explaining how the first type works and its characteristic advantage/limitation*
- *1 mark: naming the second type of system interconnection (e.g. point-to-point / switched interconnection)*
- *1 mark: explaining how the second type works and its characteristic advantage/limitation*
- *1 mark: naming the third type of system interconnection (e.g. interconnection network / multistage network)*
- *1 mark: explaining how the third type works and its characteristic advantage/limitation*
- *1 mark: naming the first CPU sub-component (e.g. Control Unit)*
- *1 mark: stating its function correctly*
- *1 mark: naming the second CPU sub-component (e.g. ALU or a named register)*
- *1 mark: stating its function correctly*

Model confidence: high.

Model note: Figure 1 is only available as extracted text; the answer does not depend on reading the figure beyond the four named components (CPU, main memory, peripherals/IO, interconnection). The question wording 'types of system interconnection' is interpreted as interconnection structures (bus, point-to-point/switched, network); alternative valid classifications (synchronous/asynchronous, centralized/distributed) are accepted in the marking. This is an unofficial model answer, not an official marking scheme.

QUESTION A8 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (6).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Computer architecture refers to the attributes of a computer system that are visible to a programmer, i.e., the conceptual design and functional behaviour of the system. It includes the instruction set architecture (ISA), data types, addressing modes, registers visible to the programmer, and I/O mechanisms. Architecture answers the question: what does the computer do? Computer organization, on the other hand, refers to the operational units and their interconnections that realize the architectural specifications. It deals with the physical implementation details such as control signals, memory technology, bus structure, and clock frequency. Organization answers the question: how is the architecture implemented? For example, whether a computer has a multiply instruction is an architectural issue; whether that multiply is implemented by a dedicated hardware multiplier or by repeated addition is an organizational issue. Architecture remains relatively stable over time, while organization may change with technology.

(b) Three CPU components and their functions:

1. Control Unit (CU): The control unit fetches instructions from memory, decodes them, and generates the timing and control signals that direct the operation of other CPU components (ALU, registers, memory, and I/O). It coordinates the execution of instructions by sequencing the micro-operations and ensuring that data is routed correctly.
2. Arithmetic Logic Unit (ALU): The ALU performs arithmetic operations (such as addition, subtraction, multiplication, and division) and logical operations (such as AND, OR, NOT, XOR, and comparisons). It takes operands from registers, performs the requested operation, and stores the result in a register, also setting condition flags (e.g., zero, carry, overflow).
3. Registers (e.g., Program Counter, Instruction Register, Accumulator, General-Purpose Registers): Registers are high-speed storage locations inside the CPU. The Program Counter (PC) holds the address of the next instruction to be fetched; the Instruction Register (IR) holds the current instruction being executed; the Accumulator or general-purpose registers hold operands and results during processing. They provide the fastest access to data and are essential for instruction execution.

Unofficial marking guide:

© Hak Cipta Universiti Teknologi MARA

• 1 mark: Correct definition of architecture (programmer-visible attributes, ISA, conceptual design).
• 1 mark: Correct definition of organization (implementation details, operational units, interconnections).
• 1 mark: Clear distinction/contrast between the two (e.g., architecture = what, organization = how).
• 1 mark: Appropriate example illustrating the difference (e.g., multiply instruction vs. implementation).
• 1 mark: Correct identification of first CPU component (e.g., Control Unit).
• 1 mark: Correct explanation of the function of the first component.
• 1 mark: Correct identification of second CPU component (e.g., ALU).
• 1 mark: Correct explanation of the function of the second component.
• 1 mark: Correct identification of third CPU component (e.g., registers).
• 1 mark: Correct explanation of the function of the third component.
Model confidence: high.

QUESTION A9 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (13).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Definition of interrupt: An interrupt is a signal (either generated by hardware or by software) sent to the CPU that temporarily suspends the currently executing program, causing the processor to save its state and transfer control to a special routine — the interrupt service routine (ISR) or interrupt handler — which services the event; when the ISR finishes, the CPU restores the saved state and resumes the interrupted program. In short, it is a mechanism by which a device or program notifies the CPU that it needs attention, allowing the CPU to respond to asynchronous events without polling.

(b) Interrupt versus direct manipulation access (direct memory access / direct program-controlled I/O): With interrupt-driven I/O, the CPU issues an I/O command and continues executing other instructions; the I/O device signals the CPU via an interrupt only when it is ready to transfer data or has completed the operation, so the CPU is freed from continuously checking the device and the transfer is asynchronous and efficient. With direct (programmed/direct manipulation) access, the CPU directly controls and manipulates the device — it repeatedly tests the device status register and moves each data item itself (polling/programmed I/O), so the CPU is tied up (busy-waiting) for the whole transfer and cannot do useful work meanwhile. Key differences: (1) initiation — interrupt I/O is device-initiated notification, direct access is CPU-driven polling; (2) CPU involvement — interrupt frees the CPU between transfers, direct access keeps the CPU occupied; (3) efficiency/speed — interrupt I/O is more efficient for slow or unpredictable devices, direct access wastes CPU cycles; (4) data transfer — in interrupt I/O the ISR handles the transfer, in direct access the CPU executes explicit load/store instructions to the device registers; (5) synchronisation — interrupt is asynchronous/event-driven, direct access is synchronous with the program loop.

(c)(i) Figure 2(a) — 'Are you sure you want to cancel this request?': this is a software interrupt (a trap / internal interrupt generated by the running program or operating system). Justification: it is raised by software (the browser/OS) to request user confirmation before performing a potentially destructive action; it is synchronous with the executing program, not caused by any external hardware device, and is handled by an OS/application service routine. Figure 2(b) — 'There is a hardware event failure in this computer. Please reboot this computer now.': this is a hardware interrupt (an external/asynchronous interrupt, here effectively a machine-check/fault condition). Justification: the message explicitly reports a hardware event failure, i.e. a signal raised by hardware (e.g. a device or the machine-check mechanism) external to the currently executing instruction stream; it is asynchronous and forces the OS to take control (prompting a reboot).

(c)(ii) Interrupt routine A is pre-empted: the CPU saves the current state (PC and registers) of routine A on the stack, suspends A, and transfers control to the higher-priority ISR for interrupt B (this is nested/priority interrupt handling). When B's routine completes, the saved state of A is restored and routine A resumes execution from the point at which it was interrupted.

Unofficial marking guide:

• 1 mark: correct definition of interrupt (signal that suspends the current program and transfers control to an ISR, resuming afterwards).
• 2 marks: correct description of interrupt-driven I/O (CPU continues, device signals when ready).
• 2 marks: correct description of direct manipulation/programmed access (CPU polls and transfers data itself, busy-waiting).

© Hak Cipta Universiti Teknologi MARA

• 2 marks: at least two valid points of difference (CPU involvement/efficiency, initiation, synchronisation).
• 2 marks: correct identification of Figure 2(a) as software interrupt/trap with justification.
• 2 marks: correct identification of Figure 2(b) as hardware interrupt with justification.
• 1 mark: correct explanation that routine A is suspended/pre-empted, its state saved, and it resumes after B completes.
Model confidence: high.
Model note: Figure 2 is only available as extracted text; the identification of interrupt types relies on the message wording. 'Direct manipulation access' is interpreted as direct/programmed (polling) I/O access, the standard contrast with interrupt-driven I/O in CSC429. This is an unofficial model answer, not an official marking scheme.

QUESTION A10 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (21).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) An I/O module is installed whenever a computer system needs to connect the CPU/memory to a peripheral device that cannot be attached directly to the system bus. It is installed at the interface point between the system bus and the external device — i.e., it is fitted when the system is configured/assembled (or when a new peripheral is added) so that the CPU can communicate with devices such as keyboards, disks, printers and network adapters. In short, it is installed whenever a peripheral must be interfaced to the processor, providing the control, data buffering and status/control signalling the CPU needs.

(b) If the I/O module is not installed, the CPU has no interface through which to exchange data with the peripheral. The device cannot be addressed or controlled, so the peripheral is effectively unusable: no data can be transferred to or from it, the CPU cannot read its status or issue commands, and any program requiring that device will fail or the device will simply not be recognised. The system loses the buffering, format conversion and error/status handling the module provides.

(c) One category of I/O module is the block-oriented (block) device module, which handles devices that transfer data in fixed-size blocks, such as disks and tape drives. Such a module typically supports direct memory access (DMA), so it can move a whole block of data between the device and main memory without the CPU handling each byte; it provides commands to read/write blocks, reports status, and generates an interrupt when the transfer completes. (Other acceptable categories: character-oriented/stream device modules for byte-at-a-time devices like keyboards and printers, or network-oriented modules.)

(d) A 'hot connection' (hot plug / hot swap) means a device or module can be connected to or removed from a running system, while it is powered and operating, without shutting the system down and without causing damage or disruption; the system detects and configures the device automatically.

(e) Three common interfaces to external devices: (1) USB (Universal Serial Bus); (2) SATA (Serial ATA) / eSATA; (3) Ethernet (network interface); also acceptable: PCIe, HDMI, FireWire (IEEE 1394), Thunderbolt, VGA.

Unofficial marking guide:

• 2 marks: (a) states the I/O module is installed at the interface between the system bus and peripheral / when a peripheral must be connected to the CPU (1 mark), and explains it provides the control/data/status interface needed for CPU–device communication (1 mark).
• 2 marks: (b) states the peripheral cannot be used / no communication with the device (1 mark), and explains why — no addressing, control, data transfer or status handling (1 mark).
• 2 marks: (c) names one valid category (e.g. block-oriented device module) (1 mark) and explains its characteristics, e.g. block transfers, DMA, interrupt on completion (1 mark).
• 1 mark: (d) correctly defines hot connection/hot plug as connecting or removing a device while the system is running/powered without shutdown or damage.
• 3 marks: (e) one mark for each of three correct common external interfaces (e.g. USB, SATA, Ethernet/PCIe/HDMI).
Model confidence: high.
Model note: OCR 'I/O' read as 'I/O'. Part (c) accepts any one valid I/O module category (block, character, or network oriented); the model answer gives block-oriented as the example. Part (e) accepts any three recognised external-device interfaces.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (20).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption on the equation (OCR correction): Figure 1's extracted text reads z=A2-B*C+jl_ D+5, which is garbled. The most conservative reading consistent with the instruction set (which has ADD, SUB, MUL, DIV but no square-root or exponent instruction) is z = (A² − B·C) / (D + 5). The jl_ is taken as a division bar /, A2 as A² (computed as A·A), and D+5 as the denominator. All three programs below compute this expression and leave the result in z. (If the intended equation differs, the same structure applies; see notes.)

Three-address program (each instruction names two sources and one destination; A, B, C, D are fixed registers, x, y, z are temporaries):

```
MUL (A, A, x)     ; x = A * A
MUL (B, C, y)     ; y = B * C
SUB (x, y, x)     ; x = A^2 - B*C   (numerator)
ADD (D, 5, y)     ; y = D + 5       (denominator)
DIV (x, y, z)     ; z = (A^2 - B*C) / (D + 5)
```

The three-address form is the most compact: one instruction per operation, no explicit data movement, because operands and result are all named in the instruction.

One-address program (an implicit accumulator; LOAD/STOR move between memory/registers and the accumulator, ADD p etc. do ACC ← ACC op p):

```
LOAD A          ; ACC = A
MUL A           ; ACC = A * A
STOR x          ; x = A^2
LOAD B          ; ACC = B
MUL C           ; ACC = B * C
STOR y          ; y = B*C
LOAD x          ; ACC = A^2
SUB y           ; ACC = A^2 - B*C
STOR x          ; x = numerator
LOAD D          ; ACC = D
ADD 5           ; ACC = D + 5
STOR y          ; y = denominator
LOAD x          ; ACC = numerator
DIV y           ; ACC = numerator / denominator
STOR z          ; z = final answer
```

The one-address form needs explicit LOAD/STOR because only one operand (the accumulator) is implicit; intermediate results must be spilled to x and y.

Zero-address program (a stack machine; PUSH p pushes operand p, POP p pops the top into p, and ADD/SUB/MUL/DIV pop two operands and push the result):

```
PUSH A          ; stack: A
PUSH A          ; stack: A, A
MUL             ; stack: A*A
PUSH B          ; stack: A*A, B
PUSH C          ; stack: A*A, B, C
MUL             ; stack: A*A, B*C
SUB             ; stack: A*A - B*C
PUSH D          ; stack: num, D
PUSH 5          ; stack: num, D, 5
ADD             ; stack: num, D+5
DIV             ; stack: num / (D+5)
POP z           ; z = final answer
```

The zero-address form uses the stack for all operands and results; note the operand order (push

© Hak Cipta Universiti Teknologi MARA

numerator first, then denominator) so that DIV computes numerator ÷ denominator.

Summary: all three programs compute z = (A² − B·C)/(D + 5) and store the final result in z; they differ only in how operands are addressed (explicitly named, via an accumulator, or via a stack).

Unofficial marking guide:

• 2 marks: correct interpretation/statement of the equation to be computed (A² − B·C)/(D + 5), including A² as A·A and D+5 as denominator

• 6 marks: correct three-address program — 1 mark each for computing A·A, B·C, the subtraction (numerator), D+5 (denominator), the division, and storing the result in z

• 6 marks: correct one-address program — 1 mark each for correct use of LOAD/STOR to form A², B·C, numerator, denominator, division, and final STOR z (accumulator semantics correct)

• 6 marks: correct zero-address program — 1 mark each for correct PUSH sequence for A², correct PUSH/MUL for B·C, correct SUB for numerator, correct PUSH/ADD for D+5, correct DIV with operands in the right order, and POP z for the final result

Model confidence: medium.

*Model note: The equation in Figure 1 is OCR-garbled ('z=A2-B*C+jl_ D+5'). I conservatively read it as z = (A² − B·C)/(D + 5), since the instruction set has no exponent or square-root operation and A² must be computed as A·A. If the actual figure shows a different expression (e.g. a different denominator or an added term), the same three program structures apply but the exact instruction sequences and mark allocation per step would change. The register names A, B, C, D are treated as fixed registers and x, y, z as temporaries, with z holding the final answer, per the question. This is an unofficial model answer, not a marking scheme.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA